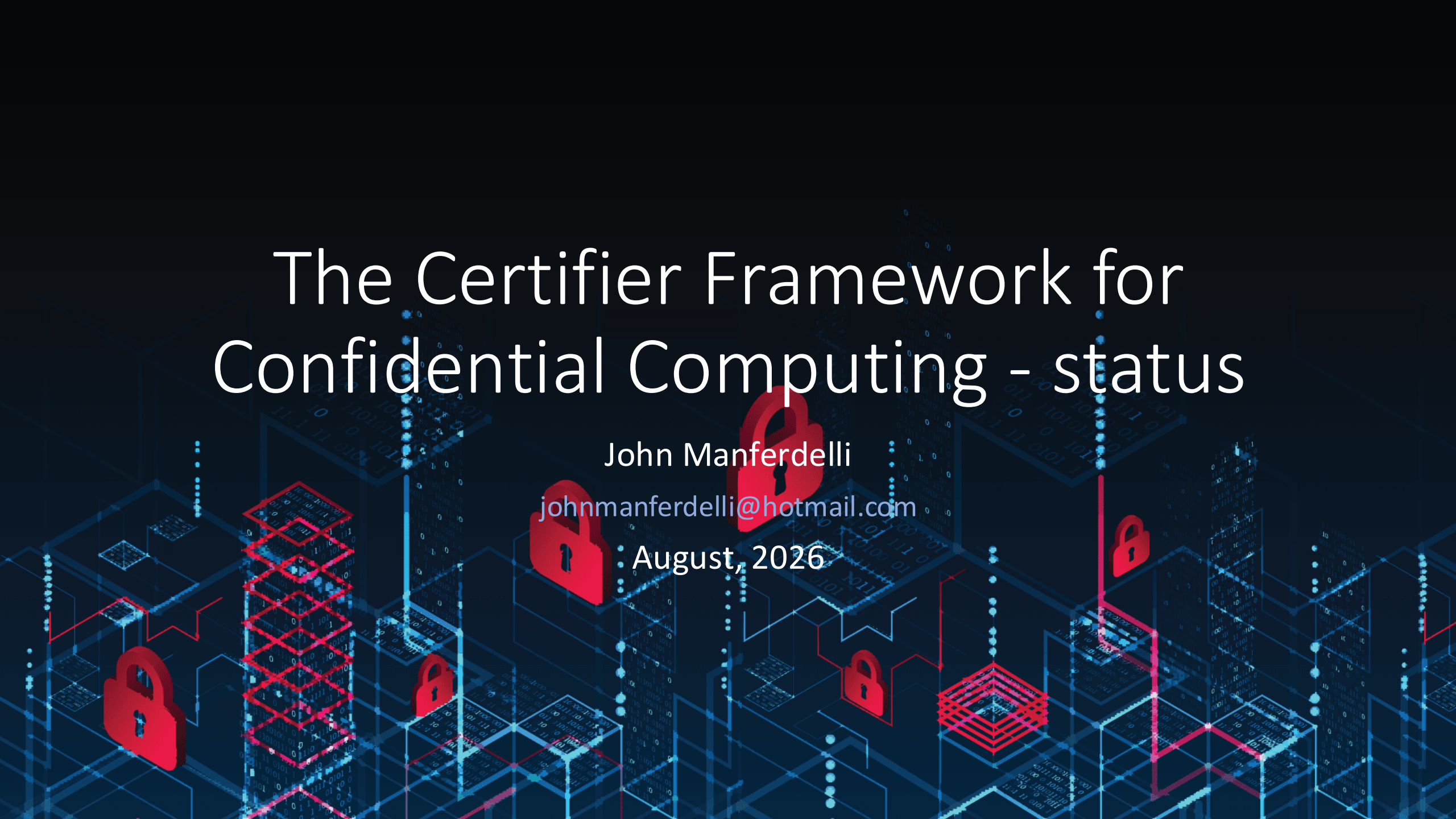
The Certifier Framework for Confidential Computing - status

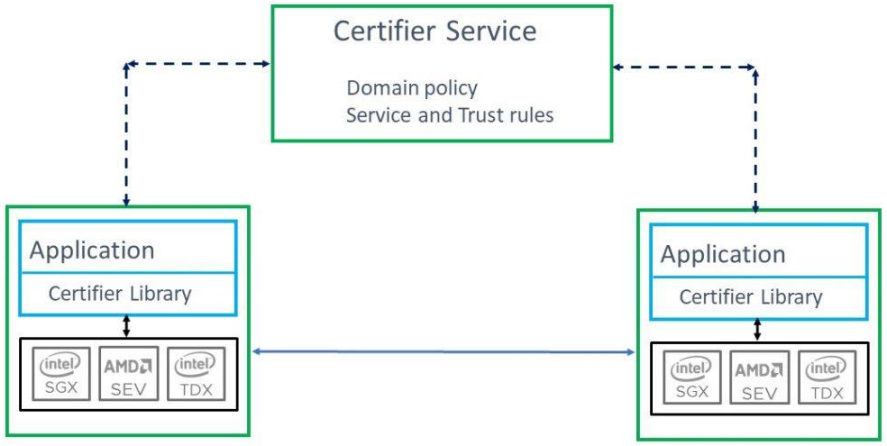
John Manferdelli

johnmanferdelli@hotmail.com

August, 2026



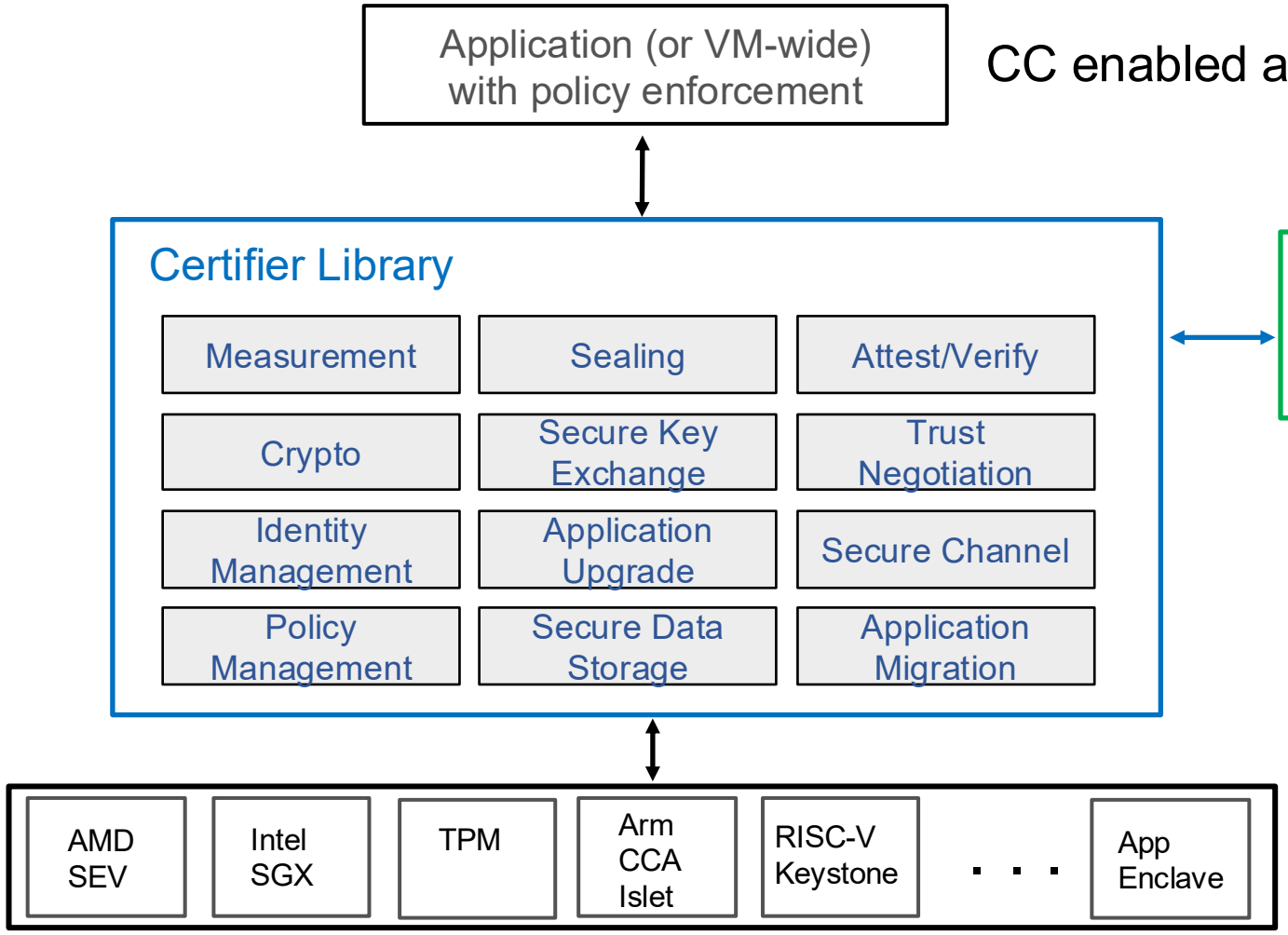
The certifier framework
Design goals



- Open-source community project
- Vastly decreases time to build applications
- Platform independent; avoid need to port applications
    - Already supports SGX (via Open Enclaves or Gramine), AMD-SEV-SNP, Arm CCA, RISC-V, TPM, Keystone, and soon TDX. Also supports recursive enclaves (applications within VM's)
    - Supports VM and application protection
- Provides simple API for most applications but complete support for "edge" cases
- Avoids dependency on a central authority or differences in deployment models
- Secure, high-availability deployment with "embarrassingly parallel" server support
- Easy to use infrastructure to support policy management
    - No deployment changes as policies evolve
    - Audited, comprehensible, declarative policy and verification
    - Efficient introduction of new components and upgrades
    - Scalable management
- Enable new services and use models for Data Economy, high security applications and regulatory regimes

Certifier Framework Architecture

Taking the devil out of the details



CC enabled app or service runs everywhere

Service runs anywhere:

- Under your desk
- In the cloud
- In a TEE or not
- At a “service provider”
- Selectable security control

CONFIDENTIAL COMPUTING CONSORTIUM

Progress since last review

Major

1. New ACL library for “granular” resource protection including web API interfaces
2. TPM support
   - Non-CC platforms as on-ramp
   - For use with cloud providers that restrict CC attestation to firmware (booting trust in v-TPM)
3. New utilities and tools for VM based applications
4. Multiple trust domain support
5. New contributor: Paul England

Plus

1. Initial Java support (still pending)
2. Bug fixes, …

Future

More platforms

- CCA, Raspberry pi, Android, Nitro(?), TDX(?), more “accelerators”
- Potential grant for app development
- Research partnerships

Other possibilities

- Rust implementation
- Prototype application
- Automated container specification, measurement and deployment
- Automated formal methods integration

Certifier Framework Concepts and API

Key Concepts:

Security Domain	Identified by a public key associated with all application code within a trusted environment. Programs are in a primary domain but can certify to secondary domains.
Certification	Refers to the verification of all properties in the security domain (including program identity, involving attestation) resulting in an x509 certificate for trust.
Trust and Policy	Should not be hard-coded in an application which should be able to operate in compliance in different security domains. Don't complicate program development or deployment.

C++ Classes:

`cc_trust_manager`	Basic interface to establish keys, policy and manage certification with the Certifier Service.
`secure_authenticated_channel`	Establishes secure channel with an "authenticated program in security domain
`policy_store`	Stores policy, keys, communications endpoints securely. Additional helper function APIs for complicated applications.

Additional helper function APIs provided for use by more complicated applications.

Simple API: Most applications will use exactly this (and nothing else)

```
string public_key_alg("rsa-2048"); string symmetric_key_alg("aes-256-gcm");
cc_trust_manager trust_mgr(“sev-enclave”, “authentication”, “store”);

trust_mgr.initialize_enclave(NULL); // init enclave
trust_mgr.init_policy_key(initialized_cert_size, initialized_cert)
trust_mgr.cold_init(public_key_alg, symmetric_key_alg, …);
trust_mgr.certify_me(); // Get admission certificate

secure_authenticated_channel channel(“client”);
channel.init_client_ssl(host, port, policy-key, auth_key, admissions-cert);

// Your application service_attestation_cert.binhere
```

Simple API: Most applications will use exactly this (and nothing else)

10 Sept 2025 We've introduced a slight API change to handle multiple domains.

```
// Initialize trust domain
string public_key_alg("rsa-2048"); string symmetric_key_alg("aes-256-gcm");
cc_trust_manager trust_mgr("sev-enclave", "authentication", "store");
trust_mgr.initialize_enclave(n, params); // init enclave
trust_mgr.initialize_store()
trust_mgr.initialize_keys(public_key_alg, symmetric_key_alg, false)
trust_mgr.initialize_new_domain(FLAGS_domain_name, purpose, serialized_policy_cert,
                                FLAGS_policy_host_url, FLAGS_policy_port))
trust_mgr.certify(FLAGS_domain_name);

// open channel
secure_authenticated_channel channel("client");
channel.init_client_ssl(FLAGS_domain_name,FLAGS_server_app_host_url,
                        FLAGS_server_app_port,trust_mgr)
// Your application here
```

Certifier: “Simple Example” App

```
Running App as server
    Client peer id is Measured
    8522d8e996e0089b26cb5dde06341c728e3017fa78974e3dce364bef56bdd307
    SSL server read: Hi from your secret client
```

```
Running App as client
    Server peer id : Measured
    8522d8e996e0089b26cb5dde06341c728e3017fa78974e3dce364bef56bdd307
    SSL client read: Hi from your secret server
```

You may want to add:

- API serialization
- ACLs for differentiated access
- Standard “distributed programming” primitives like Byzantine agreement

Works on all CC platforms:

- Simulated enclave
- AMD SEV-SNP
- Intel SGX (Gramine and OE)
- Arm CCA (Samsung Islet)
- RISC-V Keystone
- TPM
- **Application enclave service**

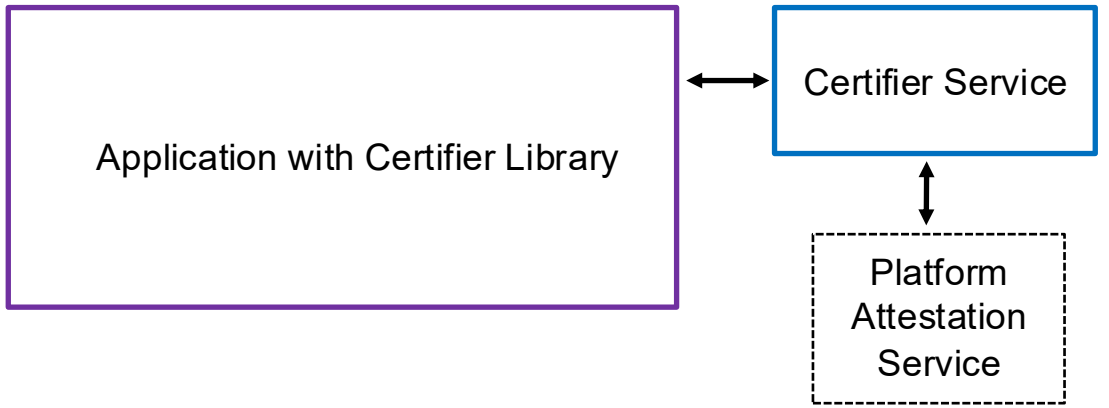
Certifier Framework API: Observations

1. **Simple** for most applications (and **adequate** for complex ones)
2. Abstracts underlying **isolate**, **measure**, **seal/unseal**, and **attest** primitives
3. Provides a **secure store** for save/recovery operations (in one statement!).
4. Includes mechanism to establish a **secure channel** within security domain
    - Encrypted, integrity protected, bi-directional, with authenticated trusted enclave named by their measurements

5. Almost as simple as **“Hello world”**

Certifier Service

"Centralized" management for a security domain



- Policy-driven (rooted in key associated with application)
- Admits new components without changing old ones
- Application upgrade without changing other applications
- Facilitates data migration and sharing in a domain
- Enforce security domain wide policy
- Machine capabilities
- Revocation

Scalable, resilient deployment supporting all environments

Policy example

```
1. Key[rsa, policyKey,
a5fc2b7e629fbbfb04b056a993a473af3540bbfe] is-
trusted

2. Key[rsa, policyKey, ... ] says
Measurement[a051a41593ced366462caea39283062874294
3c3e81892ac17b70dab6fff0e10] is-trusted

3. Key[rsa, policyKey, ...] says Key[rsa,
platformKey, ...] is-trusted-for-attestation

4. Key[rsa, platformKey, ...] says Key[rsa,
attestKey, ...] is-trusted-for-attestation
```

Join the CCC party

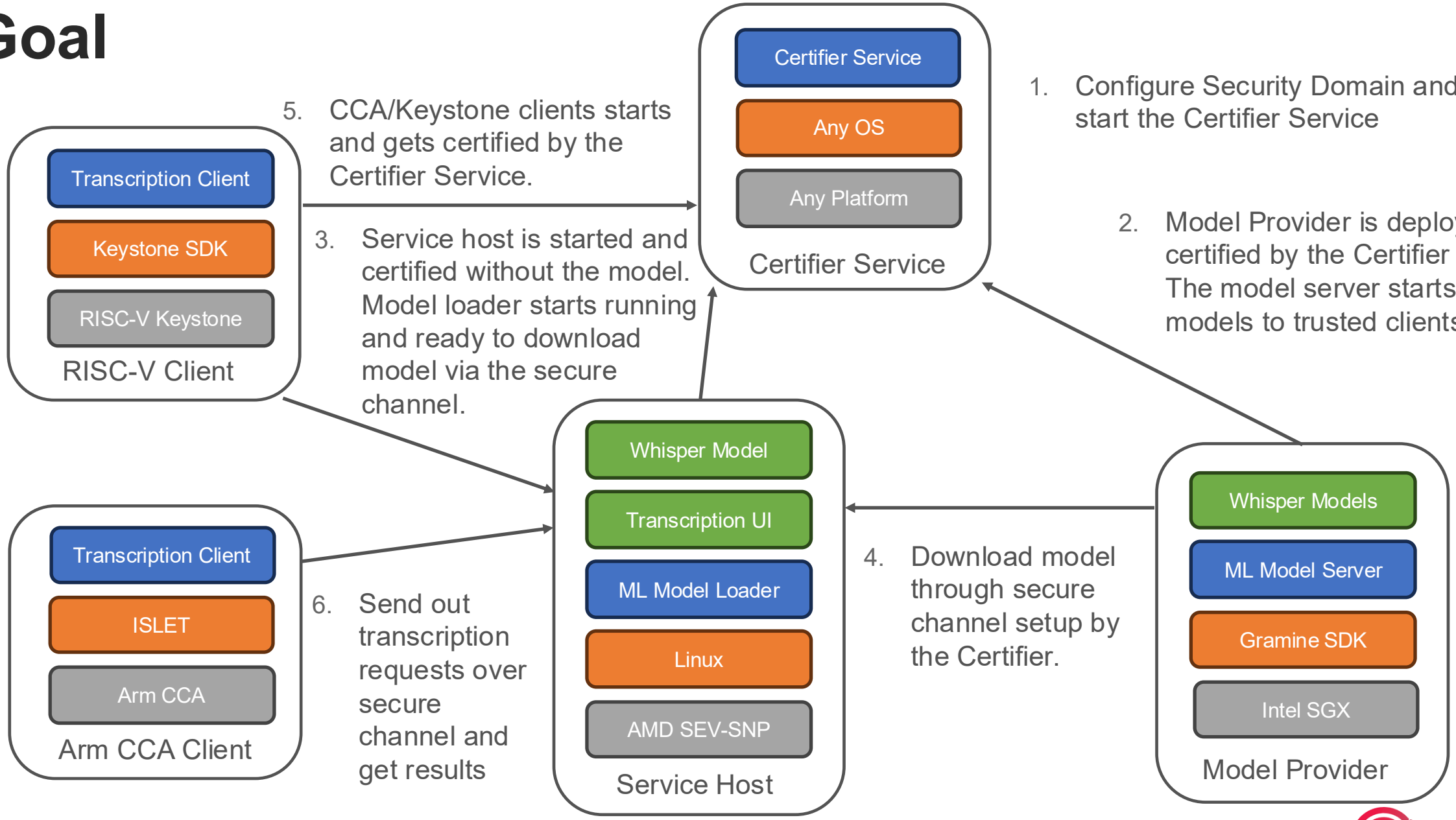
Open-Source project

- github.com/vmware-research/ccc-certifier-framework-for-confidential-computing
- Apache license
- Contributions and new contributors are welcome
- Make Confidential Computing an open Universal Platform

CONFIDENTIAL COMPUTING CONSORTIUM

Thank you!

Goal


Certifier Service
Any OS
Any Platform
Certifier Service
1. Configure Security Domain and start the Certifier Service
5. CCA/Keystone clients starts and gets certified by the Certifier Service.
Transcription Client
Keystone SDK
RISC-V Keystone
RISC-V Client
3. Service host is started and certified without the model. Model loader starts running and ready to download model via the secure channel.
2. Model Provider is deployed and certified by the Certifier Service. The model server starts serving models to trusted clients.
Whisper Model
Transcription UI
ML Model Loader
Linux
AMD SEV-SNP
Service Host
Whisper Models
ML Model Server
Gramine SDK
Intel SGX
Model Provider
4. Download model through secure channel setup by the Certifier.
Transcription Client
ISLET
Arm CCA
Arm CCA Client
6. Send out transcription requests over secure channel and get results